

Fundamentos do Tratamento de Exceções (try e except)

Objetivo: Compreender o que são erros em tempo de execução e como evitar que o programa pare de responder inesperadamente.

Conteúdos Abordados:

- Diferença entre erros de sintaxe (*SyntaxError*) e exceções em tempo de execução (*Runtime Errors*).
- Estrutura básica do bloco try e except.
- Captura de exceções específicas (ex: *ValueError*, *ZeroDivisionError*, *TypeError*).
- Captura de múltiplas exceções e uso da alias (as e).

Diferença entre erros de sintaxe (*SyntaxError*) e exceções em tempo de execução (*Runtime Errors*)

Característica	Erro de Sintaxe (<i>SyntaxError</i>)	Erro de Execução (<i>RuntimeError</i> / Exceção)
Quando ocorre?	Antes do programa começar a rodar (na fase de interpretação ou compilação).	Durante a execução do programa (o código já está rodando).
Causa	Violação das regras gramaticais da linguagem (código mal escrito).	Operação impossível ou inválida com base em dados em tempo real.
O código roda?	Não. Nenhuma linha do programa é executada.	Sim, parcialmente. O programa roda até encontrar a linha com a falha.

Estrutura básica do bloco try e except.

```
notas = {'João': [8.0, 9.0, 10.0], 'Maria': [9.0, 7.0, 6.0],  
        'José': [3.4, 7.0, 8.0], 'Cláudia': [5.5, 6.6, 8.0]  
        }
```

```
try:
```

```
    nome = input("Digite o nome do(a) estudante: ")
```

```
    resultado = notas[nome]
```

```
except Exception as e:
```

```
    print(type(e), f"Erro: {e}")
```

O try corresponde ao fluxo normal,

except ao fluxo de exceção,

try:

nome = input("Digite o nome do(a) estudante: ")

resultado = notas[nome]

except KeyError: # except ao fluxo de exceção

print("Estudante não matriculado(a) na turma")

else: # e else para o caso em que a exceção não for lançada.

print(resultado)

finally:

print("A consulta foi encerrada!")

#O finally, por sua vez, funciona para os casos em que qualquer uma das duas situações
#ocorra. Ou seja, tendo ou não exceção, a cláusula finally será executada.

Captura de exceções específicas (ex: ValueError, ZeroDivisionError, TypeError)

try:

```
x = float(input('informe um número'))
```

```
y = float(input('informe um número'))
```

```
divisao = x/y
```

except TypeError:

```
    print('erro de divisão por zero')
```

except ValueError:

```
    print('Informe apenas números')
```

except ZeroDivisionError:

```
    print('Não podemos dividir um numero por zero')
```

else:

```
    print(divisao)
```

Exercícios Propostos:

Validação de Entrada: Criar uma função que solicita a idade do usuário e trata o erro caso ele digite um texto no lugar de um número.

Calculadora Segura: Implementar uma função de divisão que trata explicitamente divisões por zero e entradas inválidas.